

Amazon ML Challenge 2026 — Team Handoff

Business Entity Resolution · state of play, measurements, and open work · 26 Sep 2026

Amazon ML Challenge 2026 — Team Handoff

Everything measured so far, what failed, what is still open, and how to get onto the working machine. Read the "Do not repeat these" section before starting — several obvious-looking ideas have already been measured and are worse.

1. Where the score is

	value
Public leaderboard	0.883
Local validation (current model)	0.92102
Expected leaderboard for current model	≈0.899
Blocking link recall	0.9251
Candidates per entity	68
Oracle ceiling on those candidates	0.96585
Top of leaderboard	0.9869

Local validation is trustworthy. The gap to the leaderboard has been a stable -0.02 across three submissions ($0.89886 \rightarrow 0.879535$, $0.90505 \rightarrow 0.883$). Tune against local validation; do not chase the leaderboard.

The oracle number is the one that should drive priorities. If the model were **perfect** on the candidates we already retrieve, we would score 0.96585. We are at 0.92102, so **~0.045 is available from a better model with no blocking change at all**. Reaching 0.985 additionally needs blocking recall above 0.97, and ours is 0.9501 even at $K=1000$ — nobody on this team has found a route there.

2. Facts about the data (measured — do not re-derive)

From the full training ground truth (2,206,821 entities, 7,638,365 links):

- **Singleton rate 5.58%** — the "does this entity match anything" gate is a minor term, not the dominant one.
- Mean **3.46** matches per entity, median 4, max 11. 85% have 2–6.
- **One-to-one holds exactly:** 7,638,365 links over 7,638,365 **distinct**

Source 2/3 ids. No record is claimed by two entities. This is the single most exploitable structural fact.

- 26% of Source 2/3 records are unmatched noise.
- US and India have statistically identical ground-truth distributions →

synthetic data, country-agnostic generator. France (test only, 15%) should follow the same priors.

- Source 1 is **100% Latin**; ~23% of Indian Source 2/3 names are in nine Indic scripts. ~3.3% of Source 2/3 addresses are empty.
- Corruption is **disjunctive**: 99.78% of true pairs have address similarity

≥70 **OR** name similarity ≥70, often not both.

- ****Co-matched records resemble each other more than they resemble their own**

Source 1 record: **max(name, address) similarity averages 97.2 over 127,802 co-matched pairs vs 46.7**** over random pairs (≥80 for 96.5% vs 0.4%).

Blocking recall by K, measured against the full 10.3M right side:

K	25	40	120	300	1000	∞ (shares ≥1 key)
recall	0.847	0.870	0.917	0.930	0.950	0.993

The true pairs *are* in the index; they are being out-ranked.

3. Do not repeat these — all measured, all worse

Idea	Result
Re-rank wide candidates by weighted-sum string similarity	0.9069 vs 0.9073 baseline
Re-rank max-dominant (wide=300 / 1000)	0.9026 / 0.9017
Score name and address as separate cosines, combined max-dominant	0.8498 vs 0.9073
Predict the whole best cluster as a hard rule	0.884 vs 0.905
Reverse retrieval to <i>*increase recall*</i>	0.856 alone; union adds only +0.008

Two lessons worth internalising:

- **IDF's rarity weighting beats plain string similarity.** Sharing the token

uneeda is strong evidence; sharing *street* is not — *token_set_ratio* counts them equally.

- **Joint evidence ranks better than disjunctive**, despite the corruption being

disjunctive, because same-street/different-business pairs vastly outnumber name-destroyed true pairs.

4. What worked (cumulative, on 60,000 held-out entities)

Change	Validation F_0.5
Baseline (K=120, 300k train entities)	0.89886
+ competition features	0.90011
+ transitivity features	0.90026
+ capacity (<i>num_leaves</i> 255; the round cap was binding)	0.90085
+ cluster-support features	0.90421
+ hybrid decision rule	0.90505
+ error-analysis fixes (see below)	0.91507
+ hybrid	0.91562
+ reverse retrieval features	0.92062
+ hybrid	0.92102

Error analysis produced more than every architectural idea combined. Three bugs found by reading actual model mistakes, worth +0.011 together:

1. **Empty addresses scored as disagreement.** `token_set_ratio("x", "")` is 0,

and address features have the highest gain, so a blank address read as "addresses strongly differ". True pairs with identical names scored 0.09. Fixed by emitting NaN (LightGBM handles missing natively).

2. **Leading zeros broke numeric matching** — `b 011 c 2` vs `b 11 c 2`.

3. **No notion of name rarity** — `blue trading` matched as confidently as

`garware educational society`. Added token-IDF features.

A second round found two more (now in the model): numeric tokens like `d1`, `c2`, `e50` were invisible because extraction only caught tokens *starting* with a digit, so `404 d1` vs `404 d6` looked identical and scored 0.999; and name-based competition ranking to rescue empty-address matches.

The single biggest feature: `rev_rank` from reverse retrieval — "is this entity this record's single best owner among all 2.2M?" It is the top feature by **5x** and lifted the singleton score 0.871 → 0.904.

5. Open work, ranked

A. Two-stage stacking — written, not yet validated

`src/stack.py` and `src/predict_stack.py` are implemented and deployed but have **never been run**. This is the highest-value open item.

The idea: cluster-support features currently anchor on `sim_block` (the blocking cosine), a weak signal, because no per-candidate probability exists at feature time. Cross-fitting fixes that — train stage 1 on 2 of 3 folds, predict the third, rotate; every training pair then has a leak-free probability, and sibling support can be weighted by it. Given the 97.2-vs-46.7 separation this should be materially stronger than the current version.

```
python -m src.stack --cands work/union_train_comp.parquet --folds 3
python hybrid.py          # retune the decision layer
# if validation improves:
python -m src.predict_stack --cands work/union_test_comp.parquet
```

~1 hour to train and measure; ~1.5 hours for test inference.

B. Close the model→oracle gap (0.921 → 0.966)

Keep mining errors — it has the best track record by far. `errors.py` on the box prints low-scored true pairs and high-scored false ones with their text. Every round so far has found something.

C. Candidate-set size is now a ranking criterion

The organisers announced mid-contest that **a smaller candidate set per entity ranks higher**, beyond the leaderboard. We went 120 → 68 per entity. The union is much more efficient per candidate:

configuration	recall	candidates/entity
forward K=120	0.9172	120.0
union K=60 + reverse top-3	0.9120	65.9
union K=40 + reverse top-3	0.9044	46.6

Worth measuring $F_{0.5}$ at K=40 — if it costs little, the smaller set is a win on a stated criterion.

D. Static multilingual embeddings

A public repo (below) uses `minishlab/potion-multilingual-128M` — *static* distilled embeddings, MIT-licensed, fast on CPU, no GPU needed. An extra retrieval channel and/or feature. Unexplored here.

6. Reference: a public repo worth reading

<<https://github.com/AyanAhmedKhan/amazon-ml-challenge>>

Independently arrived at several of our conclusions, which is reassuring: 5 sparse TF-IDF views per country partition **including a reverse target**→**S1 pass**, competition and many-to-one competition features, LightGBM, one-to-one exclusivity in the decision.

What it has that we do not:

- **Two-stage stacking** with 3-fold cross-fitting and "sibling-support

features" derived from stage-1 probabilities → this is open item A.

- Static multilingual embeddings → open item D.
- Density-matched training sampling.

Read it for methodology. Do not copy code — the organisers review the top packages in detail and the fair-play rules are explicit.

7. Getting onto the working machine

A Jarvis Labs VM (16 vCPU / 62 GB / 250 GB) holds everything: normalised parquet, candidate sets, trained models, and the generated TSVs.

- Host: `ubuntu@<current-ip>` — **the IP changes every time the instance is

paused and resumed**, so ask for the current one rather than reusing an old address.

- Key: the `aml-er.pem` private key. **It is deliberately not in this document

or the repo** — ask for it over a private channel and store it at `~/.ssh/aml-er.pem` with `chmod 600`.

- Layout on the box:

`/data/code/business_entity_resolution` (code, kept in sync with this repo), `/data/venv` (Python env), `/data/work` (parquet, candidates, models), `/data/output` (the two TSVs), `/data/student_resource` (raw data).

```
ssh -i ~/.ssh/aml-er.pem ubuntu@<current-ip>
cd /data/code/business_entity_resolution
/data/venv/bin/python -u -m src.<module>
```

Billing notes: the box costs ~\$67/hr while running and storage still bills while paused. **Pause it when idle**. Long jobs must be launched detached, or they die with the SSH session:

```
setsid nohup /data/venv/bin/python -u -m src.<module> > /data/<job>.log 2>&1 < /dev/null &
```

One trap that has bitten twice: `kill -f <pattern>` matches the SSH command line that *contains* the pattern and kills its own shell. Find the PID with `ps -eo pid,cmd | grep ... | grep -v grep` and kill by PID.

8. Working rules

- **Always measure against the full right side**. A reduced distractor pool

badly overstates recall: the same configuration scores 0.954 against a 250k pool and 0.847 against the real 10.3M.

- **Split by entity, never by pair.**
- **Report the three numbers every iteration:** recall ceiling, validation

F_0.5, singleton/non-singleton breakdown.

- **Run the official validator before every upload.** 5 submissions per day.

• Negative results are valuable — write them down. Half of what is in this document is things that did not work.

- Seeds are fixed at 42. Keep it that way so runs stay comparable.